

CITYdata Middleware Introduction and Demo

Contributors: Gabriel C. Ullmann, Sikandar Ejaz, Zongo Meyo (Minette), Rushin Makwana, Peter Yefi, Yann-Gaël Guéhéneuc

Next-Generation Cities Institute
Concordia University

February 20, 2026



Overview

1. Introduction
2. Heterogeneity and Integration Issues
3. CITYdata in a Nutshell
4. CITYdata Architecture
5. Why You Should Use CITYdata
6. How You Should Use CITYdata
7. Next Steps
8. Preparation for the Hands-on Workshop

Introduction I

- A middleware is a software that acts as a **bridge** between systems¹
- It provides **high-level building blocks** for developers²
 - **High-level** = less coding, more time to focus on the problems you want to solve and the questions you want to answer

¹<https://aws.amazon.com/what-is/middleware/>

²<https://webhost.laas.fr/TSF/IFIPWG/Workshops&Meetings/43/01-Bakken.pdf>

Heterogeneity and Integration Issues I

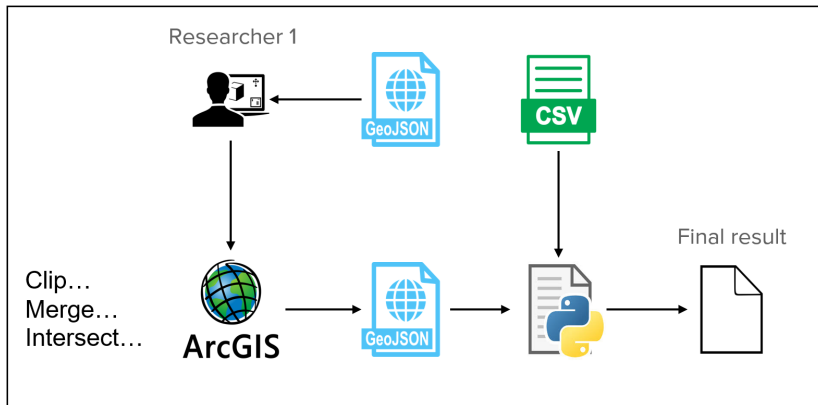


Figure: Heterogeneity Issue - Scenario 1

Heterogeneity and Integration Issues II

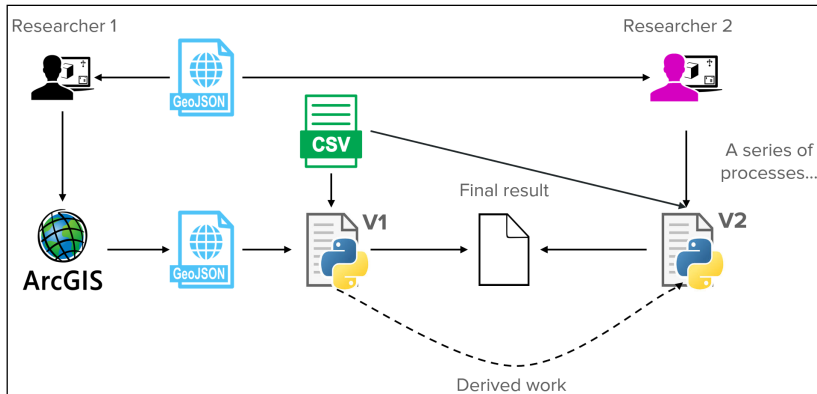


Figure: Heterogeneity Issue - Scenario 2

Heterogeneity and Integration Issues III

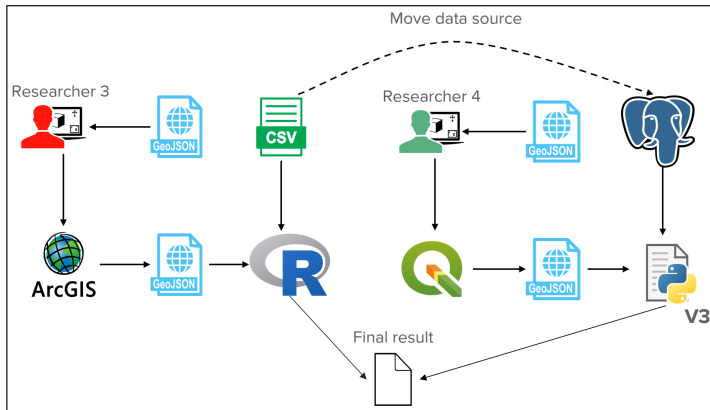


Figure: Heterogeneity Issue - Scenario 3

Heterogeneity and Integration Issues IV

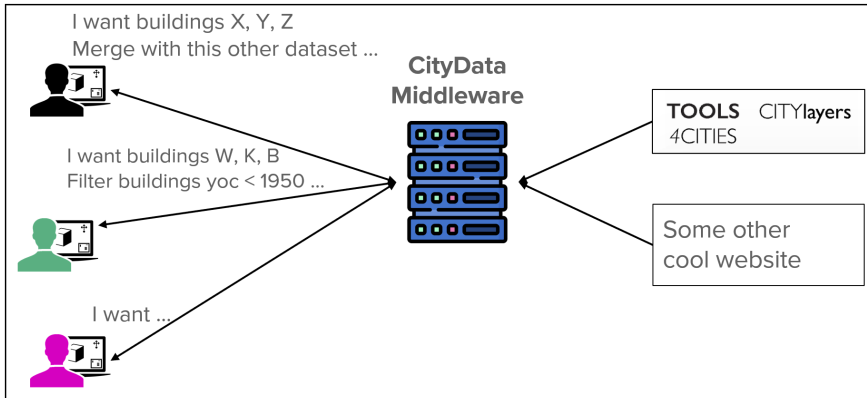


Figure: How CITYdata helps solve the problem

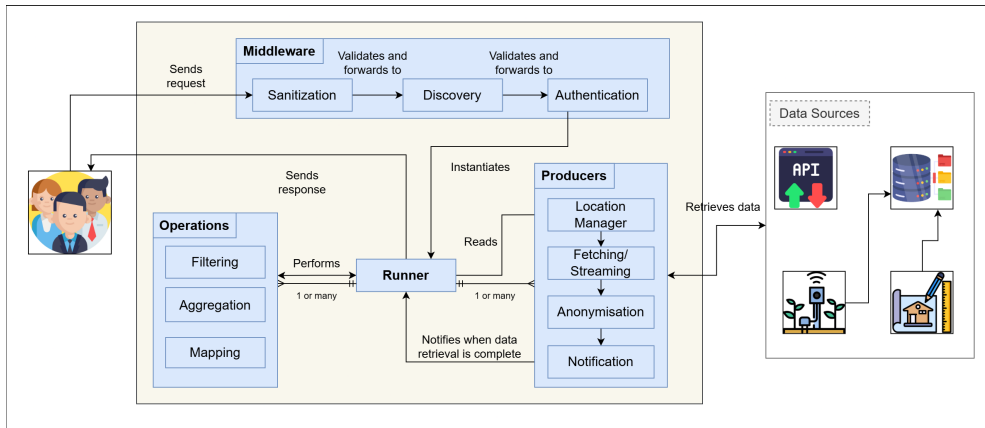
CITYdata in a Nutshell

- CITYdata³ is a middleware that allows users to **fetch**, **transform**, and **process** data from various sources using Producers and Operations
- It allows you to **integrate** data and results from **heterogeneous** sources for your research

³<https://github.com/ptidejteam/citydata>

CITYdata Architecture I

Figure: CITYdata Architecture



CITYdata Architecture II

Table: Available Producers

#	Producer Class	Parameters
1	BuildingProducer	buildingName (String), operation (IOperation), result (ArrayList), fileOptions (RequestOptions), filePath (String)
2	CKANMetadataProducer	url (String), operation (IOperation), resourceId (String), dataName (String), result (ArrayList), fileOptions (RequestOptions), filePath (String)
3	CKANProducer	url (String), resourceId (String), result (ArrayList), fileOptions (RequestOptions), filePath (String), operation (IOperation)
4	EnergyConsumptionProducer	clientId (Integer), startDatetime (String), endDatetime (String), city (String), result (ArrayList), fileOptions (RequestOptions), filePath (String), operation (IOperation)
5	EnvironmentalSensorProducer	result (ArrayList), fileOptions (RequestOptions), filePath (String), operation (IOperation)
6	GeometryProducer	operation (IOperation), city (String), result (ArrayList), fileOptions (RequestOptions), filePath (String)
7	OccupancyProducer	listSize (int), result (ArrayList), fileOptions (RequestOptions), filePath (String), operation (IOperation)
8	RandomNumberProducer	generationDelay (int), listSize (int), result (ArrayList), fileOptions (RequestOptions), filePath (String), operation (IOperation)
9	RandomStringProducer	stringLength (Integer), result (ArrayList), fileOptions (RequestOptions), filePath (String), operation (IOperation)
12	RetrofitResultsProducer	operation (IOperation), buildingIds (JsonArray), result (ArrayList), fileOptions (RequestOptions), filePath (String)
10	RoomOccupancyProducer	result (ArrayList), fileOptions (RequestOptions), filePath (String), operation (IOperation)

CITYdata Architecture III

Table: Available Operations

#	Operation Class	Parameters
1	AverageOperation	roundingMethod (String)
2	JsonArrayAverageOperation	keyName (String), roundingMethod (String)
3	JsonFilterOperation	value (String), isExactlyEqual (Boolean), key (String)
4	JsonReadOperation	path (String)
5	MergeOperation	targetProducer (String), targetProducerParams (JsonArray)
6	NullOperation	–
7	StandardAverageOperation	date (String), endTime (String), sensorIds (String), startTime (String)
8	StandardFilteringOperation	date (String), room (String), sensorId (String)
9	StringFilterOperation	filterBy (String), isExactlyEqual (Boolean)
10	StringReplaceOperation	searchFor (String), replaceBy (String)
11	TemporalAggregationOperation	date (String), room (String), endTime (String), startTime (String)

CITYdata Architecture IV

Table: REST Endpoints

#	Method	Path	Controller Method
1	GET	/authenticate	AuthController.tokenBasicAuth(HttpServletRequest)
2	GET	/health/ping	ApplicationHealthController.ping()
3	GET	/home	HomepageController.home()
4	GET	/operations/list	ListOperationsController.listOperations()
5	GET	/producers/list	ListProducerController.listProducers()
6	GET	/report/producers	ReportController.report()
7	GET	/routes/list	RouteController.listAllRoutes()
8	POST	/apply/async	ApplyController.async(String)
9	POST	/apply/sync	ApplyController.sync(String)
10	POST	/exists/	ExistsController.sync(String)

Why You Should Use CITYdata

- Reusability
- Reproducibility
- Explainability (error messages)
- Language and tool-agnostic
- Low-code (from the researcher's perspective)
- Open-source and collaborative

How You Should Use CITYdata

Demo!

Next Steps

- **START USING CITYdata!**

- Examples:

<https://github.com/ptidejteam/citydata/tree/master/Middleware/docs/examples/code>

- Queries:

<https://github.com/ptidejteam/citydata/tree/master/Middleware/docs/examples/queries>

- Connect researchers with the data that they need

- Let researchers upload their own data

- Add producers/operations that researchers often need

- Help **you** and get **your feedback**

Preparation for the Hands-on Workshop

Figure: <https://tinyurl.com/citydata-feedback>

